

NOUN-VERB ANALYSIS DOCUMENT

Movie Ticket Booking System

Course: B. Tech CSE

Semester: 5th

Subject: System Design

Subject Code: TCS-504

Assignment: Assignment 1

Introduction

Noun–Verb Analysis is used to identify the important **entities, attributes, responsibilities, and operations** of the Movie Ticket Booking System.

In this analysis:

- **Nouns** from the problem statement are considered as possible classes, attributes, or concepts.
- **Verbs** are considered as possible operations or methods.
- Each important noun is evaluated to determine whether it should become a class.
- A noun is rejected as a class when it does not represent an independent entity with its own identity, state, or responsibility.

The resulting classes and methods will be used as the foundation for the relationship table, UML class diagram, sequence diagram, and modular C++ implementation.

Nouns Identified from the Problem Statement

The important nouns identified from the problem statement are:

- Movie
- Cinema
- Customer
- Show
- Screen
- Seat
- Seat layout
- Seat type
- Booking
- Booking ID
- Payment
- Payment method
- UPI
- Card
- Cash
- Ticket
- Price

- Total amount
- Status
- Start time
- Duration
- Language
- Movie title
- Screen number
- Seat number
- Booking status

Not every noun needs to become a separate class. Some nouns represent attributes, values, states, or views of other objects.

Noun Analysis Table

Noun Found	Keep as a Class?	Decision / Reason
Movie	Yes	A movie has its own identity and data such as title, language, and duration. It is an independent entity in the system.
Cinema	Yes	The system represents a cinema that owns screens and has its own identity and data such as cinema name.
Customer	Yes	A customer is an independent entity with customer information such as name and phone number.
Show	Yes	A show represents one screening of a movie on a particular screen at a particular time and has its own state.
Screen	Yes	A screen represents one auditorium and has a screen number and physical seats.
Seat	Yes	A physical seat has its own seat number and seat type such as SILVER, GOLD, or PLATINUM.
Seat layout	No	It is a representation/view of a show's seats rather than an independent entity. It can be provided through a method of Show or related functionality.
Seat type	No	It represents a category/value of a seat rather than an independent entity. It can be represented using an enumeration such as SeatType.

Noun Found	Keep as a Class?	Decision / Reason
Booking	Yes	A booking has its own identity through a booking ID and contains booking-specific information such as selected seats, amount, and status.
Booking ID	No	It is an identifying attribute of a Booking, not an independent object.
Payment	Yes	Payment represents the abstraction/contract for processing a booking payment. It has a common operation pay(amount).
Payment method	No	It represents the type of payment being used. The actual behaviour is represented through the Payment hierarchy.
UPI	Yes	UPI represents a specific payment implementation and will be implemented as UpiPayment, derived from Payment.
Card	Yes	Card represents a specific payment implementation and will be implemented as CardPayment, derived from Payment.
Cash	Yes	Cash represents a specific payment implementation and will be implemented as CashPayment, derived from Payment.
Ticket	No	The ticket is an output representation of a confirmed booking. It does not require an independent domain class because printing is handled by TicketPrinter.
Price	No	Price is a value associated with a seat type and does not have an independent identity.
Total amount	No	It is a calculated attribute of a Booking, determined using the selected seats and their prices.
Status	No	Status represents the current state of an object, such as seat availability or booking status. It does not require an independent class.
Start time	No	Start time is an attribute of a Show.
Duration	No	Duration is an attribute of a Movie.
Language	No	Language is an attribute of a Movie.
Movie title	No	Movie title is an attribute of Movie.
Screen number	No	Screen number is an identifying attribute of Screen.

Noun Found	Keep as a Class?	Decision / Reason
Seat number	No	Seat number identifies a particular Seat and therefore is an attribute rather than a class.
Booking status	No	Booking status is an attribute/state of Booking, such as CONFIRMED, FAILED, or CANCELLED.

Important Nouns Selected as Classes

Based on the noun analysis, the following nouns are selected as classes:

Core Entity Classes

1. Movie
2. Seat
3. Screen
4. Cinema
5. Show
6. ShowSeat
7. Customer
8. Booking

Behaviour / Service Classes

9. Payment
10. UpiPayment
11. CardPayment
12. CashPayment
13. PriceCalculator
14. TicketPrinter
15. BookingService

The ShowSeat class is included even though "ShowSeat" does not appear directly as a noun in the original problem statement. It is required because the system needs to represent the **status of one physical seat for one particular show**. The assignment explicitly specifies ShowSeat as a required core entity class.

Similarly, PriceCalculator, TicketPrinter, and BookingService are service classes introduced from the required system responsibilities rather than directly extracted nouns.

Verbs Identified from the Problem Statement

The important verbs/actions identified from the problem statement are:

- Build
- See
- Play
- Pick
- Show
- Display
- Select
- Book
- Pay
- Get
- Print
- Cancel
- List
- Check
- Calculate
- Reject
- Confirm
- Release
- Become
- Support

These verbs represent actions performed by the customer or responsibilities that need to be implemented by the system classes.

Verb-to-Method Analysis

Verb / Action	Responsible Class	Possible Method	Purpose
List movies	Cinema / BookingService	listMovies()	Display currently playing movies.
Select movie	BookingService	selectMovie()	Select a movie for viewing its shows.

Verb / Action	Responsible Class	Possible Method	Purpose
List shows	Movie / BookingService	listShows()	Display shows associated with a movie.
Select show	BookingService	selectShow()	Select a show for booking.
Display seats	Show	showSeats()	Display the seat layout and availability.
Check availability	ShowSeat	isAvailable()	Determine whether a seat can be booked.
Book seat	ShowSeat	bookSeat()	Change an available show-seat to BOOKED during booking.
Release seat	ShowSeat	releaseSeat()	Make a previously booked/held seat AVAILABLE.
Calculate price	PriceCalculator	calculateTotal()	Calculate the total price of selected seats.
Pay	Payment	pay()	Define the common payment operation.
Pay using UPI	UpiPayment	pay()	Process a UPI payment.
Pay using Card	CardPayment	pay()	Process a card payment.
Pay using Cash	CashPayment	pay()	Process a cash payment.
Create booking	BookingService	createBooking()	Create a booking object for selected seats.
Confirm booking	Booking	confirm()	Change the booking status to CONFIRMED after successful payment.
Fail booking	Booking	markFailed()	Mark the booking as FAILED when payment fails.
Cancel booking	BookingService / Booking	cancelBooking() / cancel()	Cancel a confirmed booking and release its seats.
Print ticket	TicketPrinter	printTicket()	Print the details of a confirmed booking.

Verb / Action	Responsible Class	Possible Method	Purpose
Generate ID	Booking	generateBookingId()	Generate a unique booking ID using a static member.

Noun–Verb Mapping

Noun / Entity	Main Verbs / Responsibilities
Movie	Store movie information
Cinema	Own screens and provide cinema information
Screen	Own physical seats
Seat	Store seat number and seat type
Show	Represent a movie screening and display its seats
ShowSeat	Check, book, and release a seat for a particular show
Customer	Provide customer information and interact with booking operations
Booking	Store booking details, confirm, fail, and cancel
Payment	Define the payment contract
UpiPayment	Process UPI payment
CardPayment	Process card payment
CashPayment	Process cash payment
PriceCalculator	Calculate booking price
TicketPrinter	Print ticket
BookingService	Coordinate the complete booking workflow

Rejected Nouns

Some nouns identified from the problem statement are deliberately **not converted into classes**.

Seat Layout

Seat layout is rejected as a class because it does not represent an independent entity. It is simply the visual representation of the seats belonging to a particular show.

Therefore, the layout can be displayed through a method such as:

Show::showSeats()

This follows the assignment's given example.

Ticket

Ticket is rejected as a separate domain class because the assignment only requires the ticket to be **printed** after successful payment.

The responsibility for formatting and printing the ticket is therefore assigned to:

TicketPrinter

This avoids giving one class multiple unrelated responsibilities.

Seat Type

Seat Type is rejected as an independent class because SILVER, GOLD, and PLATINUM are categories of a seat rather than independent entities.

It can be represented using an enumeration:

- SILVER
- GOLD
- PLATINUM

The corresponding prices can then be handled by PriceCalculator.

Price

Price is rejected as a class because it is a value associated with a seat type.

The pricing responsibility belongs to:

PriceCalculator

Booking ID

Booking ID is rejected as a class because it is simply a unique identifier stored within a Booking.

Start Time

Start Time is rejected as a class because it is an attribute of a Show.

Duration

Duration is rejected as a class because it is an attribute of a Movie.

Language

Language is rejected as a class because it is an attribute describing a Movie.

Status

Status is rejected as a class because it represents the state of another object.

For example:

- ShowSeat → AVAILABLE / BOOKED
- Booking → CONFIRMED / FAILED / CANCELLED

Final Class List

After completing the noun–verb analysis, the final class set for the system is:

Core Entity Classes

1. **Movie**
2. **Seat**
3. **Screen**
4. **Cinema**
5. **Show**
6. **ShowSeat**
7. **Customer**
8. **Booking**

Payment Classes

9. **Payment** — abstract base class
10. **UpiPayment**
11. **CardPayment**
12. **CashPayment**

Service Classes

13. **PriceCalculator**
14. **TicketPrinter**
15. **BookingService**

This class list matches the classes specified by the assignment.

Final Method Candidates

The initial method candidates identified through noun–verb analysis are:

Movie

- getTitle()
- getLanguage()
- getDuration()

Seat

- getSeatNumber()
- getSeatType()
- getPrice()

Screen

- addSeat()
- getScreenNumber()
- getSeats()

Cinema

- addScreen()
- addMovie()
- getMovies()
- getScreens()

Show

- getMovie()
- getScreen()
- getStartTime()
- showSeats()
- getShowSeats()

ShowSeat

- isAvailable()
- bookSeat()
- releaseSeat()

- getSeat()
- getStatus()

Customer

- getName()
- getPhone()

Booking

- getBookingId()
- getShow()
- getSeats()
- getTotalAmount()
- getStatus()
- confirm()
- markFailed()
- cancel()

Payment

- pay()

UpiPayment

- pay()

CardPayment

- pay()

CashPayment

- pay()

PriceCalculator

- calculateTotal()

TicketPrinter

- printTicket()

BookingService

- bookTicket()
- cancelBooking()
- listMovies()
- listShows()

Conclusion

The Noun–Verb Analysis identifies the entities and actions required to implement the Movie Ticket Booking System.

The analysis distinguishes between:

- **Entities** that require classes,
- **Attributes** that belong inside classes,
- **States** that represent object conditions,
- **Views/concepts** that do not require independent classes, and
- **Actions** that become class methods.

The final design contains **15 classes**, consisting of core entity classes, payment classes, and service classes. This class structure will be used consistently in the next stages of the system design.

The noun–verb analysis therefore provides the foundation for determining each class's **attributes, methods, responsibilities, relationships, and access modifiers** in the next design stage.